

Flight-Controller Receive Hardening

What changed in the CATIA UART receive path, why it matters on the F4 autopilot, how it was tested, and what remains unproven.

System: Adam F4 and Adam NPS

Date: 21 September 2026

Status: Implemented and validated locally

The patch closes two concrete failure mechanisms: an invalid frame length could underflow inside the shared CATIA parser, and fixed-layout FC handlers could consume a checksum-valid frame with the wrong payload size. Validation demonstrates that malformed lengths are now rejected without changing autopilot camera state.

4

focused source and test files changed

0..250

payload lengths exercised for both fixed-layout replies

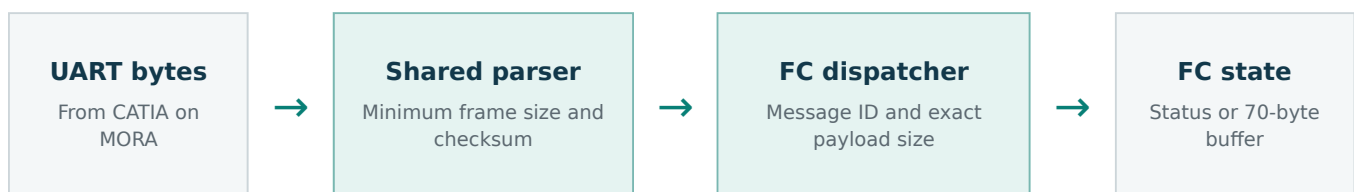
4

simulated camera cycles completed without stopping NPS

Result in one sentence: malformed CATIA status and image-payload frames can no longer make the FC copy data unless their payload length exactly matches the message contract.

System Boundary

CATIA runs on the MORA companion computer. The Paparazzi digital-camera module runs on the F4 flight controller, or inside NPS during local simulation. Both environments compile the same parser and FC dispatch implementation.



CATIA frames contain an STX marker, total length, message ID, payload, and two checksum bytes. The minimum legal total size is therefore five bytes. A valid checksum proves that bytes arrived consistently; it does **not** prove that a message has the correct semantic size.

SODA is downstream of capture. CATIA starts SODA after image capture and EXIF processing. SODA does not directly transmit UART or UDP replies to the FC. The immediate ordinary UART response after accepting a shot is CATIA status.

Failure Mechanisms and Fixes

1. Reject impossible frame lengths before subtraction

catia/protocol.c

The parser previously calculated `payload_len = length - 5` without first proving that `length` was at least five. Because the value is unsigned, lengths zero through four could wrap into a large apparent payload and poison frame synchronization.

BEFORE

Impossible length accepted; unsigned subtraction could wrap; later UART bytes could be consumed as a bogus payload.

AFTER

Any length below `CatiaSizeOf(0)` is rejected before arithmetic; the parser error count increments and its state resets.

2. Enforce the status message contract

uart_cam_ctrl.c

`CATIA_STATUS` has an eight-byte fixed layout. The FC handler previously copied that fixed layout after checking only the message ID. It now requires `payload_len == CATIA_STATUS_MSG_SIZE` before changing `digital_cam_uart_status`.

BEFORE

A short but checksum-valid status frame could cause bytes beyond the received payload to enter FC status state.

AFTER

Every non-eight-byte status frame is ignored; only the exact protocol layout can mutate status state.

3. Enforce the 70-byte payload contract

uart_cam_ctrl.c

`CATIA_PAYLOAD` carries exactly 70 bytes. The FC handler now requires `payload_len == CATIA_PAYLOAD_MSG_SIZE` before copying into thumbnail storage.

BEFORE

The handler always copied 70 bytes, even when the current frame declared a smaller or larger payload.

AFTER

Wrong-sized frames leave thumbnail storage untouched; an exact 70-byte frame is still accepted.

Why These Boundaries

- **The parser owns framing.** Rejecting an impossible total length there prevents bad arithmetic and protects every parser user.
- **The consumer owns message meaning.** Exact status and payload sizes belong in the FC dispatcher because it knows each fixed-layout contract.
- **Checks are layered.** Framing and checksum validation cannot replace message-specific validation.
- **Behavior remains compatible.** Correctly encoded eight-byte status and 70-byte payload messages follow the original path unchanged.

Regression-Test Structure

Parser boundary tests

catia/tests/pose_protocol_test.c

The test sends every impossible total-frame length from zero through four. Each case must be rejected, return the state machine to its initial state, and increment the parser error count.

Actual FC dispatch tests

catia/tests/pose_sender_test.c

A new helper constructs complete CATIA frames with valid checksums and feeds them byte by byte through `digital_cam_uart_event()`. This tests the production FC implementation rather than a duplicate model.

- **Status rejection:** every representable payload length from 0 through 250, except eight, must leave FC status unchanged.
- **Status acceptance:** an exact eight-byte frame must update the tested shot count to 321.
- **Payload rejection:** every representable payload length from 0 through 250, except 70, must leave thumbnail storage unchanged.
- **Payload acceptance:** an exact 70-byte frame must be copied byte for byte.

Validation Evidence

Validation	What it demonstrates	Result
CATIA host test variants	Parser and real FC event-path regression cases execute successfully.	Passed
ASan and UBSan	Exhaustive hostile-length execution produced no detected memory or undefined-behavior fault.	Passed
Adam <code>ap.compile</code>	The hardened code compiles for the embedded F4 autopilot target.	Passed
Adam <code>nps.compile</code>	The same autopilot receive implementation compiles in the simulator.	Passed
Four-shot local integration	NPS sent four AI-camera triggers; CATIA captured four mock images, wrote EXIF, ran SODA, and returned status without stopping NPS.	Passed
Live 70-byte telemetry round trip	The direct handler is exhaustively tested, but the optional live buffer request was not observed in the integration run.	Not observed

What the Result Means

Demonstrated: the most concrete FC-side malformed-frame hazards found in this investigation are removed. Correct frames remain accepted, wrong-sized frames are ignored, both target builds pass, and repeated local camera activity did not stop the simulated autopilot.

Not demonstrated: this does not prove that either defect caused the physical F4 failure. The aircraft and MORA hardware were unavailable, no reset-cause registers or hard-fault dump were captured, and the complete live 70-byte thumbnail telemetry path was not exercised end to end.

Recommended hardware follow-up

1. Repeat the shot sequence on the bench with the aircraft disarmed.
2. Record F4 reset-cause flags and any hard-fault state before normal startup clears them.
3. Capture both UART directions, including declared frame lengths and checksums.
4. Exercise the optional 70-byte buffer request and verify the returned frame end to end.
5. Compare behavior with and without SODA processing while keeping CATIA status traffic identical.

Source checkout: `~/paparazzi` resolves to `~/develop/allthings_paparazzi/paparazzi_main/`; these are the same repository. This note describes the focused modifications in `protocol.c`, `uart_cam_ctrl.c`, `pose_protocol_test.c`, and `pose_sender_test.c`.